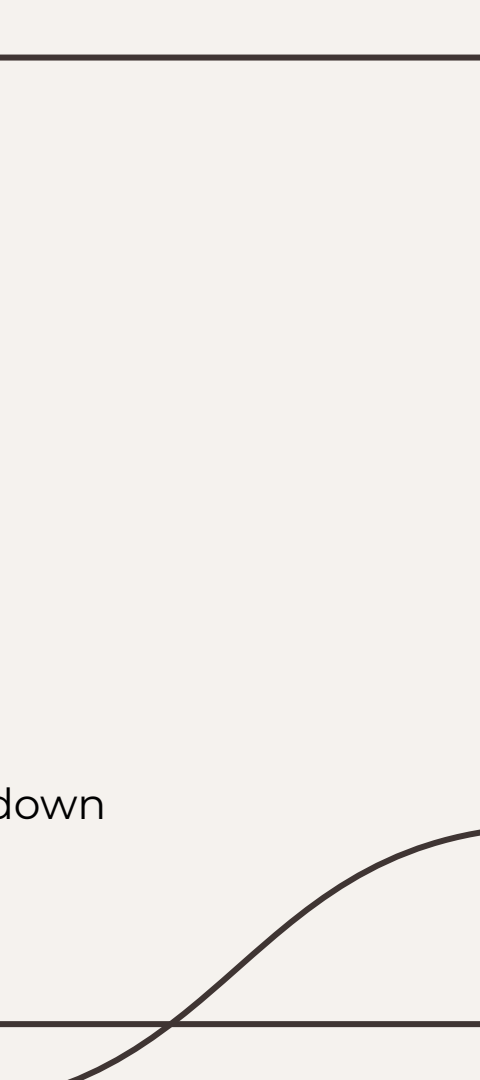


# Présentation Microdown

Ronan, Aymen, Maria

Github : <https://github.com/Aymenmessikh/Microdown>



---

# Sommaire

**01**

**Design Microdown**

**02**

**Nouvelle Fonctionnalité**

**03**

**Design Hider**

**04**

**Point Intéressant**

---

# Design avant : Problème actuel

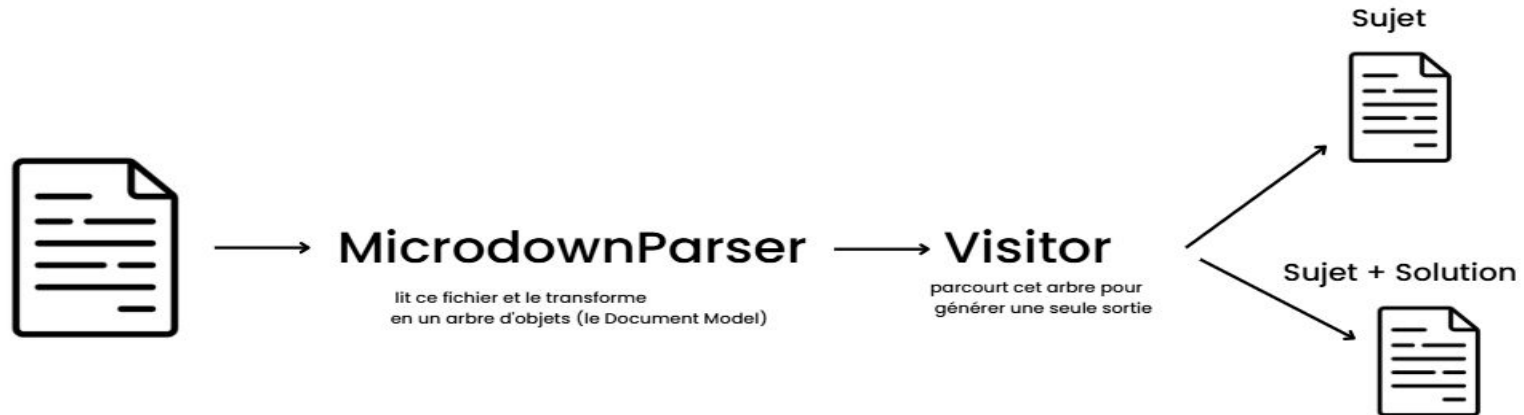
Microdown est un moteur de transformation conçu pour du "1 pour 1".



Ce système fonctionne très bien pour de la documentation standard

# Design avant : Problème actuel

- L'Objectif : Une source unique, deux fichiers de sortie



impossible avec le Visitor actuel

# Design avant : Architecture existante

01

02

03

## Le parser Microdown:

- Transforme le Markdown en un modèle d'objets (Document Model).
- Reconnaît les blocs de code, listes, titres...
- **Ne fait rien de spécial pour les exercices.**

## Les Visitors:

- Parcourent le document pour produire : HTML, PDF, LaTeX...
- **Ils n'ont aucun comportement pour masquer les corrections.**

## Les Builders textuels:

**MicMicrodownTextualBuilder**  
pour construire du texte

**MicOutputStream** pour écrire  
proprement dans un flux

**MicMinimalConfiguration** pour  
gérer les retours à la ligne

Aucun Builder ou Visitor n'est dédié aux exercices.

---

# Nouvelles Fonctionnalités:

## 1-Séparer automatiquement les exercices et les solutions :

- Microdown mélange tout dans un seul fichier.
- Nous voulons deux fichiers distincts : **un pour l'étudiant, un pour les solutions.**

## 3-Automatiser tout le processus :

- Microdown doit reconnaître les blocs **exercice**, masquer la solution, et créer un fichier **Solutions.md** automatiquement.

## 2-Faciliter la création de documents pédagogiques :

Permettre aux professeurs de produire facilement :

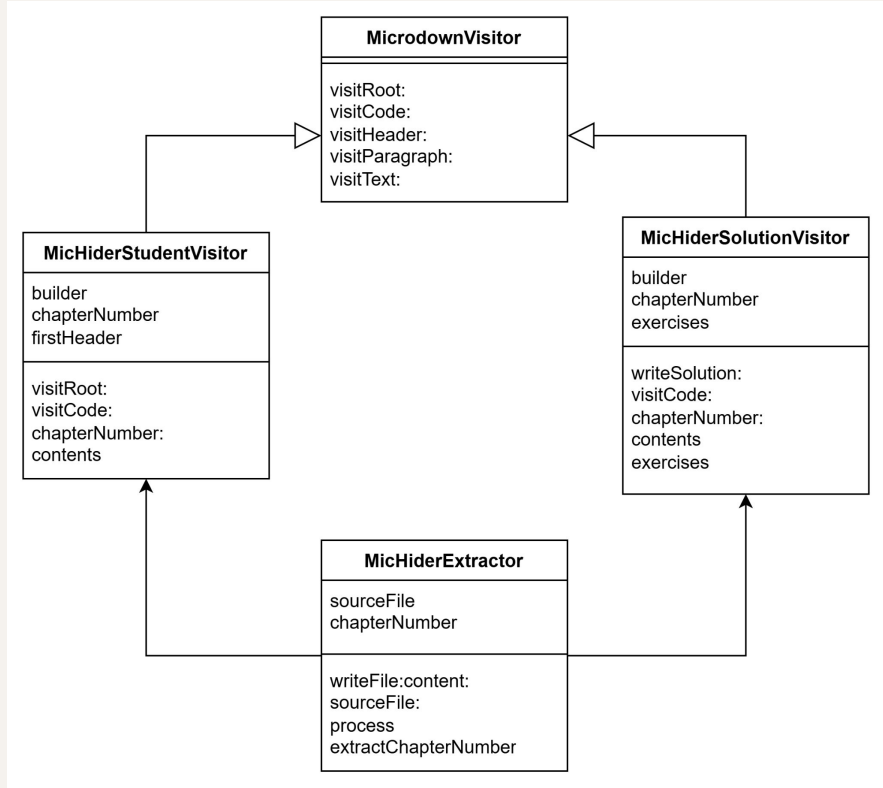
- des documents sans solution.
- un fichier de corrigés.

## 4-Améliorer Microdown avec deux Visitors spécialisés :

- Un Visitor = fichier étudiant
  - Un Visitor = fichier solutions
  - > Modularité, propreté, extensibilité.
-

# Design Hider : Architecture global

- Extractor = gestion du flux
- Parsing Microdown
- Deux visiteurs séparés
- Pattern utilisé : **Visitor**
- Output :
  - File1.md (étudiant)
  - File1Solutions.md (solutions)



# Design Hider : Student Visitor

- But : montrer l'exercice sans la correction
- Analyse chaque exercice du fichier
- Garde seulement l'en-tête de l'exercice
- Remplace le contenu par un placeholder

```
```exercice&anchor=exfoobar
    Foo >> bar
    some code
    more code
    even more code
    ```
```

```
## Chapt
```

```
@Chap1
```

```
```exercice
    Foo >> bar
    ... Your code ...
    ```
```

# Design Hider : Solution Visitor

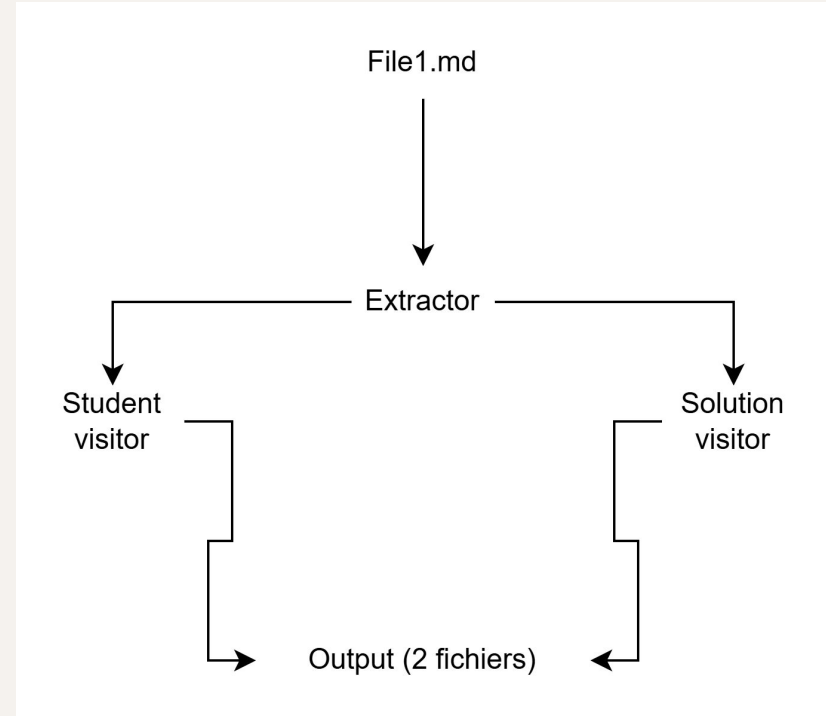
- But : regrouper toutes les solutions
- Parcourt les mêmes exercices
- Récupère le code complet
- Ajoute une phrase de contexte (ancree + chapitre)
- Produit un fichier contenant toutes les réponses, dans l'ordre

```
```exercise&anchor=exfoobar
    Foo >> bar
    some code
    more code
    even more code
    ```
```

```
Here is the solution of the code *@exfoobar@* in Chapter *@Chap1@*
```
Foo >> bar
    some code
    more code
    even more code
```
```

# Design Hider : Extractor et Visitors

- Une seule analyse du fichier : deux traitements
- Flux clair
- Faible couplage entre les composants



# Point Intéressant : Numéro de chapitre

- Le nom du dossier contient "ChapterX"
- Le programme extrait le numéro automatiquement
- Utilisé dans les deux fichiers générés
- Évite les risques d'erreurs manuelles
- Rend le système réutilisable pour n'importe quel chapitre

# Conclusion

- Deux visiteurs : deux rôles complémentaires
- Automatisation complète du traitement
- Architecture claire et modulable
- Résultats utilisable en contexte réel
- Projet facilement extensible